Access control	An IAM policy with secretsmanager:GetSecretValue, and optionally a resource policy on the secret.
Audit	Every read and change is recorded in AWS CloudTrail.
Replication	A secret can be copied to other Regions for disaster recovery.
Point	**Secrets Manager vs. KMS**
What it holds	Secrets Manager: secret values such as passwords and API keys. KMS: cryptographic keys that encrypt and decrypt, whose material never leaves KMS.
Main calls	Secrets Manager: GetSecretValue and PutSecretValue. KMS: Encrypt, Decrypt, and GenerateDataKey.
Rotation	Secrets Manager: rotates the secret value on a schedule. KMS: rotates the key material, once a year when it is turned on.
Azure equivalent	Secrets Manager: Key Vault secrets. KMS: Key Vault keys. Certificates go to AWS Certificate Manager.

S3 encryption	**Simple meaning**
SSE-S3	S3 manages the keys. It is the default for every new object, and it costs nothing extra.
SSE-KMS	A KMS key encrypts the objects, so key use is logged and can be controlled. Readers need kms:Decrypt as well as s3:GetObject.
SSE-C	The caller supplies the key with every request. AWS does not store it.

SESSION 2 — AWS II: PIPELINE BUILDING AND MONITORING

- **Topic:** AWS II — Pipeline Building and Monitoring
- **Date:** 2nd November 2026

5. AWS Step Functions: Fundamentals and Workflow Orchestration

In simple words. A data pipeline is rarely one task. It is a chain of steps: start something, wait for it, check the result, and do something else if it failed. AWS Step Functions is the serverless service that runs such a chain for you. You describe the steps once, and Step Functions runs them in order, keeps track of where each run is, retries what failed, and records everything. This job is called workflow orchestration.

Term	**Simple meaning**
State machine	The workflow itself, written as a JSON document in the Amazon States Language (ASL). It lists the states and how they connect.
State	One step in the state machine, such as running a job, making a choice, or waiting.
Execution	One run of a state machine. Each execution has its own input, output, status, and history.
Workflow Studio	The visual editor in the console. It lets you drag states onto a canvas and writes the JSON for you.
Execution role	The IAM role that the state machine uses to call other services. Its trust policy names states.amazonaws.com (section 1).
State type	**What it does**
Task	Does the real work by calling a service, such as starting a Glue job, running a Lambda function, or sending an SNS message.
Choice	Looks at the data and picks the next state, like an if and else.
Parallel	Runs several branches at the same time and waits for all of them.
Map	Repeats the same steps for every item in a list, such as each table or each file.
Wait	Pauses for a set time or until a given moment.
Pass	Passes its input on, optionally changing it. It is useful for testing.
Succeed and Fail	End the execution as a success or as a failure, with an error name and a cause.

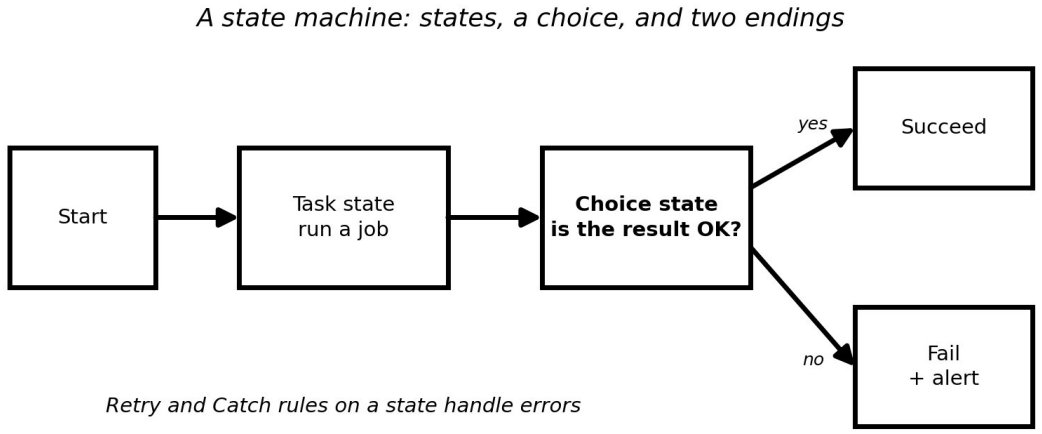


Diagram 6: A state machine moves from state to state, and each Task can have retry and catch rules.

Waiting for a job. A Task state can call a service in three ways, and the ending of its Resource name tells which one.

Pattern	**Simple meaning**
Request and response (default)	Step Functions calls the service and moves on as soon as the call is accepted. It does not wait for the work to finish.
Run a job (.sync)	Step Functions starts the job and waits until it has really finished, then moves on. Glue, ECS, and Batch work this way.
Wait for callback (.waitForTaskToken)	Step Functions hands a token to another system and pauses until that system sends the token back. It is used for approvals and slow outside systems.
Workflow type	**Simple meaning**
Standard	Runs for up to a year, runs each step exactly once, and keeps the execution history for 90 days. It is billed per state transition. This is the choice for data pipelines.
Express	Runs for up to five minutes at a very high rate, and is billed by the number of executions and their duration. History goes to CloudWatch Logs. It suits short event processing.
Way to start	**Simple meaning**
Console or API	A person or program calls StartExecution and passes a JSON input.
EventBridge schedule	A rule or a scheduler starts the state machine at set times, such as every day at 02:00. It plays the role of a trigger.

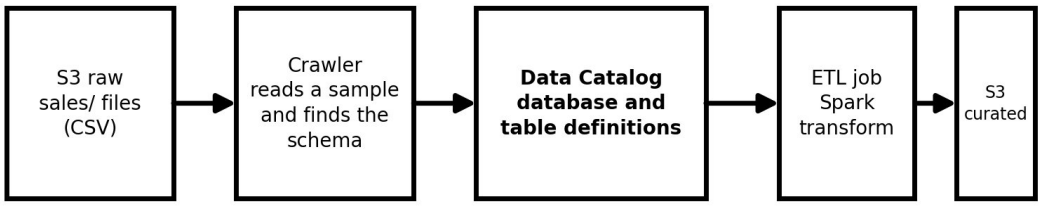
EventBridge event	An event, such as a new file in S3, starts the state machine.
Another service	Services such as API Gateway or another state machine can start an execution.

6. AWS Glue: Crawlers, Data Catalog, and ETL Jobs

AWS Glue is the serverless data integration service of AWS. It finds out what data you have, records it in a catalog, and runs the code that cleans and moves it. There are no clusters to create. Glue runs the work on managed Apache Spark, and you pay only while the work runs.

Component	**Simple meaning**
Data Catalog	The central list of metadata: databases and tables that say where the data is, which columns it has, and in which format. It holds the description of the data, and the data itself stays in S3.
Crawler	A program that looks at a data store, reads a sample of the data, works out the schema, and creates or updates the table in the Data Catalog.
Classifier	The rule a crawler uses to recognize a format, such as CSV, JSON, or Parquet. Built-in ones cover the common formats.
Connection	The saved details for reaching a data store such as a JDBC database. The password can be kept in Secrets Manager (section 4).
ETL job	The code that extracts data, transforms it, and loads it somewhere else. It is written in PySpark or Scala, or built in the visual editor.
Glue Studio	The visual editor and notebook environment where jobs are drawn or written and then tested.

AWS Glue: from raw files to a clean table



The catalog stores the schema and the location of the data, and not the data itself

Diagram 7: A crawler records the schema in the Data Catalog, and the ETL job uses that table.

The Data Catalog. A Glue database is only a named group of tables, and a table is a definition: the S3 location, the format, the columns, and the partitions. Services such as Amazon Athena, Redshift Spectrum, EMR, and Glue jobs all read the same catalog, so the schema is described once and used everywhere. Databricks can also use it as a metastore. Access to databases and tables can be controlled in more detail with AWS Lake Formation.

Crawler setting	**Simple meaning**
Data source	Where to look, such as the S3 path s3://tricloud-datalake-dev-001/raw/sales/, a JDBC database, or DynamoDB.
IAM role	The role the crawler uses. It needs read access to the data and the Glue service permissions, so give it only the prefixes it needs.
Target database	The Glue database that will hold the tables the crawler makes.
Schedule	On demand, hourly, daily, or a custom cron. A crawler can also be started from a pipeline.
Schema change policy	What to do when the data changes: update the table, or only log the change. It also decides what happens to tables whose data was removed.

Partitions. If files are stored in folders such as raw/sales/year=2026/month=10/, the crawler turns year and month into partition columns. Queries that filter on them read only the matching folders, which is faster and cheaper.

Job type	**Simple meaning**
Spark	The main type. It runs PySpark or Scala code on managed Spark for large data.
Spark Streaming	Reads from a stream, such as Kinesis or Kafka, and

	processes it continuously.
Python shell	Runs plain Python on a small machine. It suits small tasks such as moving a small file or calling an API.
Job setting	**Simple meaning**
IAM role	The role the job runs with. It reads the input, writes the output, and may read a secret, so it needs those permissions and no more.
DPU and worker type	The size of the job. A DPU (Data Processing Unit) is a measure of compute power. Choose a worker type and a number of workers.
Job bookmark	A memory of what the job has already processed, so that the next run takes only new data.
Job parameters	Values given when the job starts, written as --name value, and read in the script with getResolvedOptions.
Glue version	The version of Spark and Python that the job uses.

Step	**What to do**
1. Create the role	Create an IAM role that trusts glue.amazonaws.com. Attach the AWSGlueServiceRole managed policy and a scoped S3 policy like the one in section 2.
2. Create a database	In the Glue console create a database such as tricloud_raw.
3. Create the crawler	Point it at s3://tricloud-datalake-dev-001/raw/sales/, give it the role and the database, and choose a schedule.
4. Run the crawler	After the run, a table named sales appears in the database, with its columns and partitions.
5. Create the job	In Glue Studio, read the catalog table, add the transform, and write Parquet to the curated prefix.
6. Run and check	Run the job, then open the job run to see its status, its duration, and its logs in CloudWatch.

7. Step Functions and Glue Together in a Data Pipeline

Glue does the data work, and Step Functions decides the order and handles the problems. Step Functions starts the crawler, waits for it, starts the Glue job with the right parameters, waits for it to finish, and sends an alert if anything fails. Neither service has to know the details of the other.

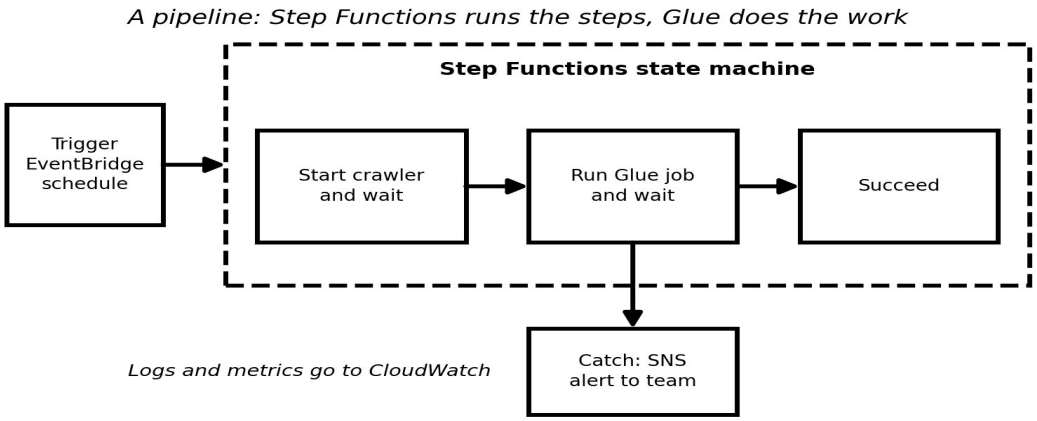


Diagram 8: A schedule starts the state machine, which runs the crawler and the job and alerts on failure.

Step	What happens
1. Start	An EventBridge schedule starts an execution every night and passes the run date as input, such as {"run_date": "2026-11-02"}.
2. Crawl	A Task state starts the crawler. There is no ready-made wait for crawlers, so the state machine loops: Wait, then GetCrawler, then a Choice that checks whether the state is READY.
3. Run the job	A Task state with glue:startJobRun.sync starts the job, passes --run_date, and waits until the run is finished.
4. Check	A Choice or the job result decides whether to continue, for example to a second crawler for the curated table.
5. Alert	A Catch rule sends any error to an SNS topic, and the execution ends as Fail so that it shows as failed.